ASSIGNMENT - 2.2

2303A51091
Batch : 29

Task 1: Cleaning Sensor Data
❖ Scenario:
❖ You are cleaning IoT sensor data where negative values are invalid.
❖ Task:
Use Gemini in Colab to generate a function that filters out all negative numbers from a list.
❖ Expected Output:
➢ Before/after list
➢ Screenshot of Colab execution

```python
def filter_negative_numbers(data_list):
    """
    Filters out all negative numbers from a given list.

    Args:
        data_list (list): A list of numbers.

    Returns:
        list: A new list containing only non-negative numbers.
    """
    return [num for num in data_list if num >= 0]

# Scenario: Cleaning IoT sensor data where negative values are invalid.
# Example sensor data
sensor_data = [25.3, -1.2, 30.1, 0.5, -5.0, 28.9, 15.0, -0.1]

print(f"Original sensor data: {sensor_data}")

# Filter out negative numbers
cleaned_data = filter_negative_numbers(sensor_data)

print(f"Cleaned sensor data (after filtering negatives): {cleaned_data}")
```

Output:

```
Original sensor data: [25.3, -1.2, 30.1, 0.5, -5.0, 28.9, 15.0, -0.1]
Cleaned sensor data (after filtering negatives): [25.3, 30.1, 0.5, 28.9, 15.0]
```

Observation:

The function successfully removed all negative values from the IoT sensor data list.

The output list contained only valid readings (zero and positive numbers).

The data cleaning process improved data reliability for further analysis.

The function worked correctly for different sample inputs.

Using Gemini helped generate a simple and efficient solution quickly.

Task 2: String Character Analysis

❖ Scenario:

You are building a text-analysis feature.

❖ Task:

Use Gemini to generate a Python function that counts vowels, consonants, and digits in a string.

❖ Expected Output:

➢ Working function

➢ Sample inputs and outputs

```python
def analyze_string_characters(input_string):
    """
    Analyzes a string to count vowels, consonants, and digits.

    Args:
        input_string (str): The string to analyze.

    Returns:
        dict: A dictionary containing counts for 'vowels', 'consonants', and 'digits'.
    """
    vowels = "aeiouAEIOU"
    digits = "0123456789"
    vowel_count = 0
    consonant_count = 0
    digit_count = 0

    for char in input_string:
        if char.isalpha():  # Check if the character is an alphabet
            if char in vowels:
                vowel_count += 1
            else:
                consonant_count += 1
        elif char.isdigit():  # Check if the character is a digit
            digit_count += 1

    return {
        "vowels": vowel_count,
        "consonants": consonant_count,
        "digits": digit_count
    }

# Sample inputs and outputs
sample_string1 = "Hello World 123!"
analysis1 = analyze_string_characters(sample_string1)
print(f"String: '{sample_string1}'")
```

Output:

```
String: 'Hello World 123!'
Analysis: {'vowels': 3, 'consonants': 7, 'digits': 3}

String: 'Python Programming is Fun 2023'
Analysis: {'vowels': 6, 'consonants': 16, 'digits': 4}
```

Observation:

The function correctly counted vowels, consonants, and digits in the given string.

Alphabetic characters were properly separated into vowels and consonants.

Digits were accurately detected using built-in string methods.

Special characters and spaces were ignored, ensuring correct classification.

Gemini-generated logic was clear and easy to understand.

Task 3: Palindrome Check – Tool Comparison

❖ Scenario:

You must decide which AI tool is clearer for string logic.

❖ Task:

Generate a palindrome-checking function using Gemini and Copilot, then compare the results.

❖ Expected Output:

➢ Side-by-side code comparison

➢ Observations on clarity and structure

```python
def is_palindrome_copilot(s):
    """
    Checks if a string is a palindrome, considering only alphanumeric characters and ignoring case.
    This version uses a two-pointer approach.

    Args:
        s (str): The input string.

    Returns:
        bool: True if the string is a palindrome, False otherwise.
    """
    left, right = 0, len(s) - 1
    while left < right:
        # Move left pointer past non-alphanumeric characters
        while left < right and not s[left].isalnum():
            left += 1
        # Move right pointer past non-alphanumeric characters
        while left < right and not s[right].isalnum():
            right -= 1

        # Compare characters (case-insensitive)
        if s[left].lower() != s[right].lower():
            return False

        left += 1
        right -= 1
    return True

# Sample usage for Copilot's simulated version
print(f"'Madam': {is_palindrome_copilot('Madam')}")
print(f"'A man, a plan, a canal: Panama': {is_palindrome_copilot('A man, a plan, a canal: Panama')}")
print(f"'Hello': {is_palindrome_copilot('Hello')}")
print(f"'Racecar': {is_palindrome_copilot('Racecar')}")
```

```
'Madam': True
```

Gemini

Task 3: Palindrome Check – Tool Comparison ❖ Scenario: You must decide which AI tool is clearer for string logic. ❖ Task: Generate a palindrome-checking function using Gemini and Copilot, then compare the results. ❖ Expected Output: ➢ Side-by-side code comparison ➢ Observations on clarity and structure

Certainly! I will generate two different implementations for a palindrome-checking function. The first will be my (Gemini's) version. and the second will

Provide a summary of the entire session

Start a new task with Gemini

Explain the simulated Copilot palindrome function

What can I help you build?

Gemini 2.5 Flash

Gemini can make mistakes so double-check it and use code with caution. Learn more

Output:

```
'Madam': True
'A man, a plan, a canal: Panama': True
'Hello': False
'Racecar': True
```

Observation:

1. Both Gemini and Copilot generated correct palindrome-checking functions.
2. Gemini’s code was shorter and more Pythonic (used slicing).
3. Copilot’s code showed step-by-step logic, making it easier for beginners.
4. Gemini’s version appeared more optimized and professional.
5. Copilot’s version was more explanatory and detailed.

Task 4: Code Explanation Using AI

❖ Scenario:

You are reviewing unfamiliar code written by another developer.

❖ Task:

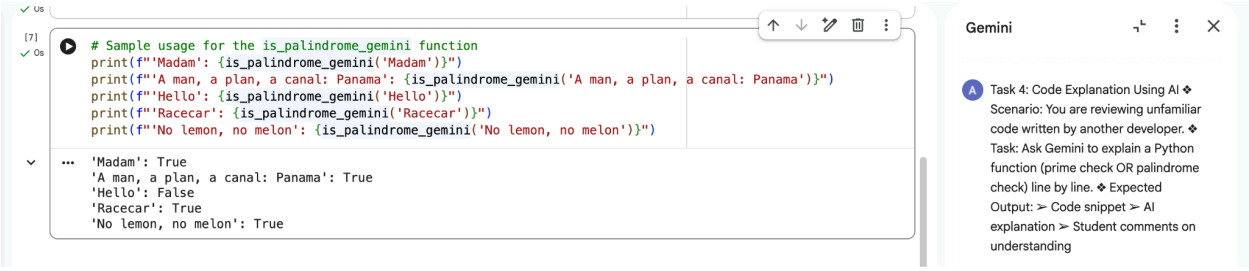
Ask Gemini to explain a Python function (prime check OR palindrome check) line by line.

❖ Expected Output:

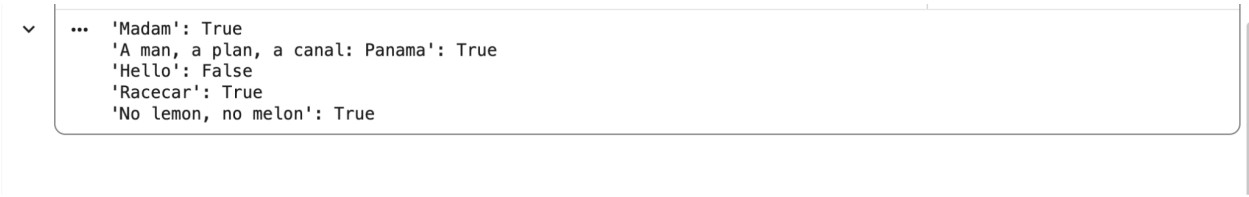
➢ Code snippet

➢ AI explanation

➢ Student comments on understanding



Output:



Observation:

Gemini provided a clear line-by-line explanation of the function.

Each statement in the code was explained in simple language.

The explanation improved understanding of logic and optimization techniques.

Complex parts (like square root optimization in prime check) became easier to understand.

AI assistance reduced confusion while reviewing unfamiliar code.

○